```
// Clutch Alert worker: runs every minute on Cloudflare, checks NBA games,
// and sends a push through ntfy when a game is close in the final minutes.
//
// Setup values (Worker > Settings):
//   Variable  NTFY_TOPIC  required, a long random topic name only you know
//   Variable  MARGIN      optional, points apart that counts as close (default 5
//   Variable  WINDOW_SEC  optional, seconds left in the 4th quarter or OT (defau
//   KV binding  SEEN      required, remembers which games already alerted

const ESPN = "https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreb

function parse(ev) {
  const comp = (ev.competitions && ev.competitions[0]) || {};
  const st = ev.status || comp.status || {};
  const type = st.type || {};
  const g = {
    id: ev.id,
    state: type.state || "pre",
    period: st.period || 0,
    clock: Number(st.clock) || 0,
    display: st.displayClock || "",
    home: { name: "TBD", score: 0 },
    away: { name: "TBD", score: 0 },
  };
  for (const c of comp.competitors || []) {
    const side = c.homeAway === "home" ? g.home : g.away;
    side.name = (c.team && (c.team.abbreviation || c.team.shortDisplayName)) || "
    side.score = Number(c.score) || 0;
  }
  return g;
}

function isClutch(g, margin, windowSec) {
  return (
    g.state === "in" &&
    g.period >= 4 &&
    g.clock <= windowSec &&
    Math.abs(g.home.score - g.away.score) <= margin
  );
}

function periodLabel(p) {
  return p <= 4 ? "Q" + p : p === 5 ? "OT" : p - 4 + "OT";
}
```

```
async function push(env, title, body) {
  const res = await fetch("https://ntfy.sh/" + encodeURIComponent(env.NTFY_TOPIC)
    method: "POST",
    headers: { Title: title, Priority: "high", Tags: "basketball" },
    body,
  });
  if (!res.ok) throw new Error("ntfy returned " + res.status);
}

async function check(env, { test = false } = {}) {
  if (!env.NTFY_TOPIC) throw new Error("NTFY_TOPIC is not set");
  if (test) {
    await push(env, "Clutch Alert test", "If you can read this, pushes work.");
    return { test: "sent" };
  }
  const margin = Number(env.MARGIN) || 5;
  const windowSec = Number(env.WINDOW_SEC) || 180;

  const res = await fetch(ESPN, { headers: { accept: "application/json" } });
  if (!res.ok) throw new Error("ESPN returned " + res.status);
  const data = await res.json();
  const games = (data.events || []).map(parse);

  const sent = [];
  for (const g of games) {
    if (!isClutch(g, margin, windowSec)) continue;
    const key = "alert:" + g.id + ":" + g.period;
    if (await env.SEEN.get(key)) continue;
    const diff = Math.abs(g.home.score - g.away.score);
    const title = g.away.name + " " + g.away.score + ", " + g.home.name + " " + g
    const body =
      (diff === 0 ? "Tied" : "Within " + diff) + " with " + g.display + " left in
    await push(env, title, body);
    await env.SEEN.put(key, "1", { expirationTtl: 6 * 60 * 60 });
    sent.push(title);
  }
  return { checked: games.length, live: games.filter((g) => g.state === "in").len
}

export default {
  async scheduled(event, env, ctx) {
    ctx.waitUntil(check(env));
  },
  // Open the worker's URL to run a check by hand. Add ?test=1 to send a test pus
  async fetch(request, env) {
    try {
```

```
      const test = new URL(request.url).searchParams.get("test") === "1";
      return Response.json(await check(env, { test }));
    } catch (e) {
      return Response.json({ error: String(e.message || e) }, { status: 500 });
    }
  },
};
```